NiCE Task to Budget Sync: Rebuild with Error Handling

The three-recipe chain is replaced by two small recipes plus a scheduled replay recipe: budget writes become minimal diffs, SF cases are created by upsert, and every failure is parked in an error table and retried, so repeat events and replays never create duplicates.

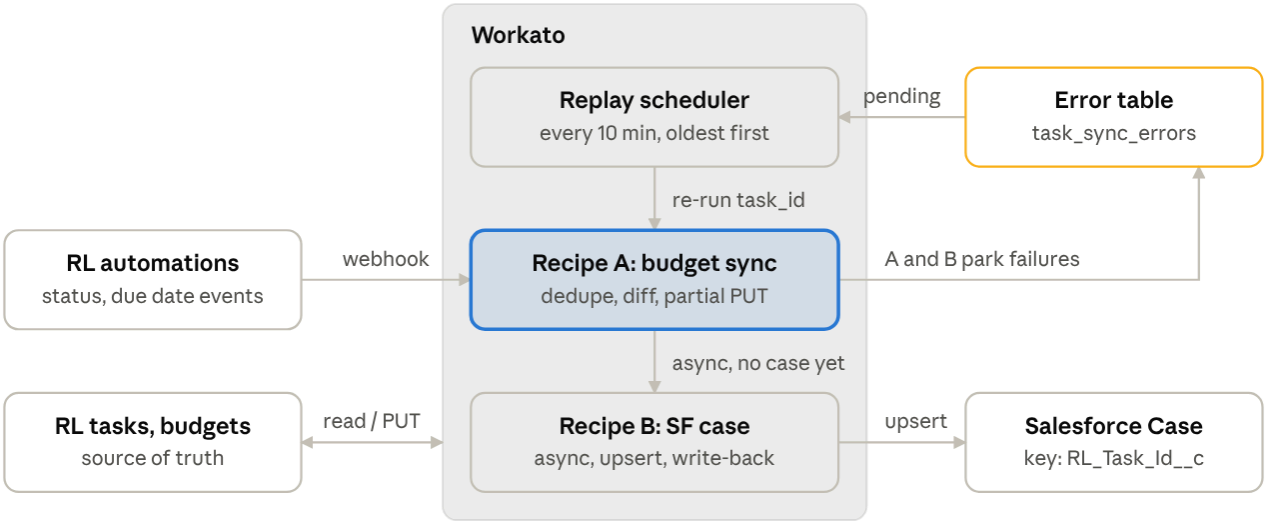
Proposed Approach

Every status or due date change runs a 3-recipe chain at concurrency 1 with no retries.

Problem	Impact
Steps 13 to 26 search every milestone task for counts that are never sent	Biggest API load; drives the rate limit issue
Step 33 runs with wait for response off and sends a full PUT	Failures go unseen; may clear Synced to Oracle; overwrites concurrent edits
Fixed 60 second wait before case creation	Blocks the queue for every completed milestone
Case created before the case ID is written back, no retry	Failed write-back causes a duplicate case next event
Do-not-retry on all errors; Go_Live_Date__c set to today; hardcoded IDs	Transient 429/5xx become manual work; wrong dates on reprocess; hard to maintain

Architecture

Two recipes and a replay scheduler, with failures parked in one table



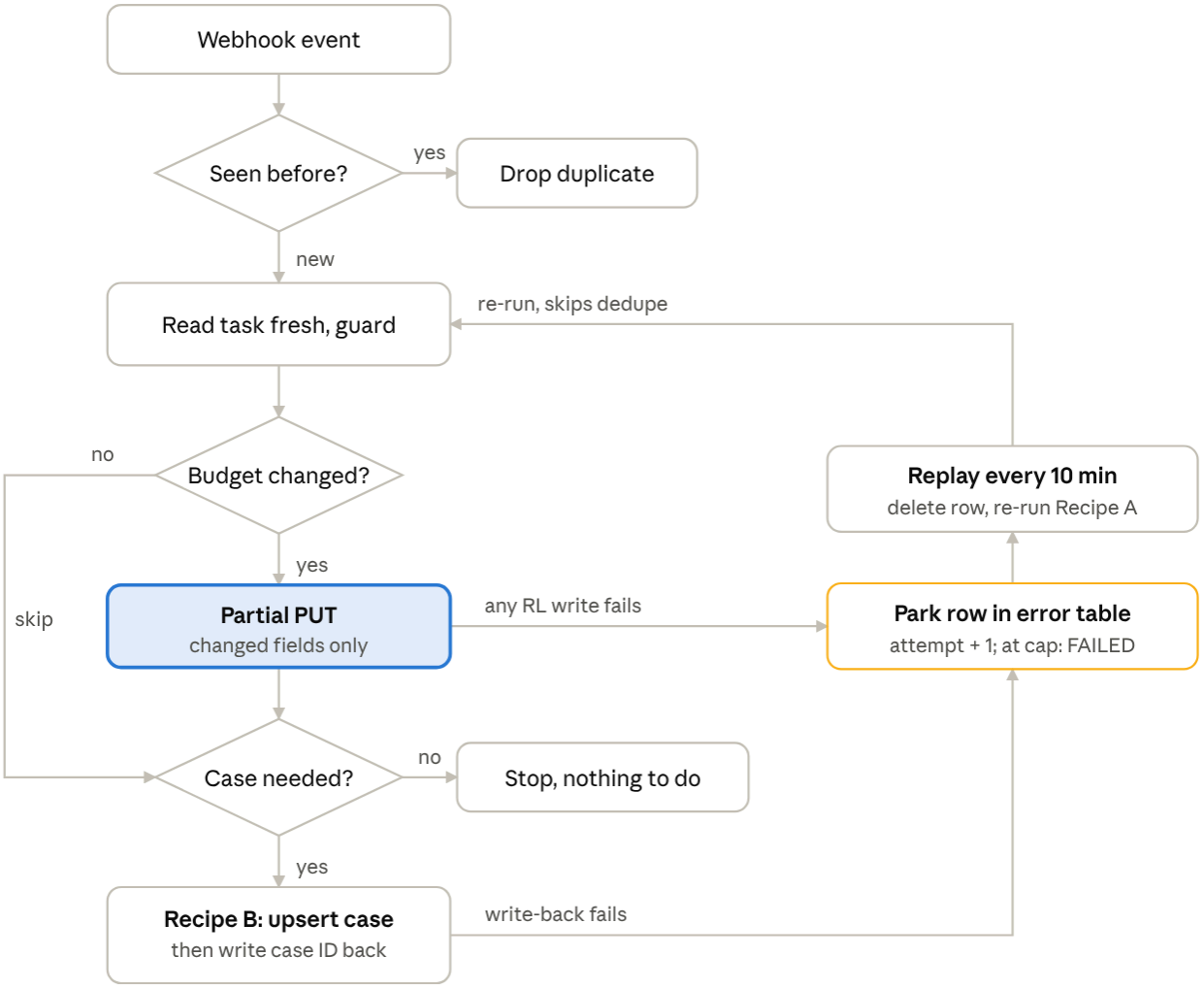
Data tables also hold event keys (24h TTL) and the config table (IDs, owners, field IDs).

architecture · 3 recipes, 3 systems

Rocketlane sends one webhook to Recipe A. Recipe A is the only entry point: it writes budgets, hands case creation to Recipe B, and parks any failure in the error table, which the scheduler replays through Recipe A.

Data flow

One event writes only what changed, and any failure is parked and replayed



data flow · one event, 3 decisions, 1 failure loop

A case is needed when the task is a completed Go-live or Billing Milestone with no case number. Replays enter at Read task, so they skip event dedupe.

Recipes

Recipe A: budget sync (webhook or replay, concurrency 1)

1. Dedupe the event (webhook only): key `task_id | trigger_type | status | due_date | updatedAt`, save with 24h TTL, stop if seen.
2. Search Task once; stop if no budget IDs or the milestone type is not relevant.
3. Compute target values in one Python step; field IDs come from the config table.
4. Per budget ID: GET that budget, skip unless FIXED_FEE and MILESTONE_BASED, diff, partial PUT only changed fields.

5. If a completed Go-live or Billing Milestone has no case number, call Recipe B async.
6. Log one line: task ID, budgets touched, fields changed.

Recipe B: SF case (async, concurrency 1)

1. Re-fetch the task; exit if a case number exists.
2. Upsert the Case on `RL_Task_Id__c`; Go_Live_Date__c = Completed At; owner, record type, business hours, subject and product from the config table.
3. Write the case ID to the task, then to each linked budget (partial PUT).
4. Create Case Product Allocations; collect failures into one alert.

Replay scheduler (every 10 minutes): select PENDING rows oldest first (batch size N), delete the row, re-run Recipe A inside a Monitor, short Wait between rows. A failure re-inserts the row with attempt + 1. A daily summary alert lists rows by age and status.

Error handling: park, replay

No in-job retries, so the concurrency 1 queue never blocks. Wrap every RL write in a Monitor; the Catch inserts a row in `task_sync_errors` and the job ends cleanly. Replays are safe because Recipe A reads fresh and writes only the diff.

- **One row per task:** update the existing row instead of inserting a second.
- **Attempt cap (e.g. 5):** on the last attempt set FAILED, which the scheduler ignores, and send one alert.
- **Crash window:** deleting before the re-run can lose a row if the scheduler dies mid-run; delete after success closes the gap (decision open).
- **Table columns:** task_id, recipe, step, http_status, error_message, payload (original webhook), attempt, status (PENDING or FAILED), created_at.

Error	Handling
429, 5xx, timeout	Park; usually succeeds on a later run
409/412 (concurrent edit)	Park; the fresh read on re-run resolves it
400/404/422 (data error)	Park; fails each time, reaches the cap, FAILED and alert
401/403 (auth)	Park and alert on the first failure; do not wait for the cap

Duplicate protection: three layers

Replays are safe because running twice gives the same result: one budget write and one case, however many times an event or replay runs.

Layer	Where	Stops
1. Event dedupe	Recipe A entry (webhook only)	Same event delivered twice
2. Diff check	Recipe A	Budget writes when nothing changed
3. Case upsert	Recipe B	Duplicate SF cases after a failed write-back

Layer 1: only exact repeats drop; a real status change has a new key.

```
T-123|status|COMPLETED|10:00:04   new, processed
T-123|status|COMPLETED|10:00:04   duplicate, stopped
T-123|status|IN_PROGRESS|10:05:00 new, processed
```

Layer 2: the Due date event finds nothing left to write.

```
Event 1: diff = {status: COMPLETED, actual_date: 2026-10-12}  -> PUT only
these
Event 2: diff = {}                                            -> no PUT

def diff(target, current):
    return {k: v for k, v in target.items() if current.get(k) != v}
```

Layer 3: a failed write-back no longer creates a second case.

```
OLD: Event 1 creates Case 500-A, write-back fails
     Event 2: no case number -> creates Case 500-B (duplicate)
NEW: Event 1 upserts on RL_Task_Id__c = T-123 -> Case 500-A, write-back fails
-> row parked
     Replay: upsert finds T-123 -> returns 500-A, write-back succeeds
```

Old vs new, rollout, open items

Situation	Current	Proposed
API calls per event	Task searches per config per budget, GET all budgets, full PUT each	1 task search, up to 1 GET and 1 PUT per linked budget, only on change
Latency per completed milestone	60 second blocking wait	No fixed wait
RL update fails	Manual backlog work	Parked and replayed every 10 min
Same event twice	Runs twice	Dropped
Case write-back fails	Duplicate case	Upsert returns existing case
Permanent data error	Sits in backlog, no limit	Attempt cap, FAILED, one alert

Rollout: build in a sandbox with fixtures (every status, both milestone types, multi-budget tasks, injected 429/5xx/4xx) then shadow run in parallel logging would-be writes, then switch per automation keeping old recipes stopped for rollback, then retire after a stable period.

Open items for discussion

1. RL webhook delay affects event dedupe. The dedupe key includes `updatedAt` from the payload, so a delayed or out-of-order delivery looks like a new event and passes. That is safe for budgets (Recipe A reads fresh and the diff finds nothing to write) but the dedupe then saves no work, and a delay longer than the 24h TTL would reprocess an old key.

- Treat dedupe as an optimisation only; correctness comes from the fresh read and the diff.
- Proposed: store `last_processed_updatedAt` per task and drop any event older than it.
- To confirm: typical and worst-case RL webhook delay, and whether 24h TTL is long enough.

2. Parametrized backlog rows, so an old queued row cannot overwrite a newer record. A row parked days ago must not push old values onto a task that has changed since.

- Store only parameters in the error table (task_id, recipe, step, trigger_type), never field values; the replay re-derives every value from the current task.

- One pending row per task: a newer event for the same task updates the row, and a successful run deletes it.
- Replay re-checks the task before writing, so a stale row ends as a no-op diff.
- To confirm: whether the `payload` column is kept for triage only, not for the re-run.